MERIDIAN: Latent Cartographer - Foundation Test Cases

15 verified test cases for Milestone 1 Foundation covering all implemented browser extension modules.

Project name	MERIDIAN: Spatial Cartographer	Class / roll no.	3P11/ 1024160014 3P11/ 1024160133 3P11/ 1024160130	App version	0.1.0 (Milestone 1 Foundation)	Environment	Chromium MV3 / ONNX Runtime WASM / macOS / Node 22	Execution date	2026-11-04

Test case ID	Module	Scenario / objective	Test type	Preconditions	Test data	Test steps	Expected result	Actual result	Status	Priority	Evidence / defect ID
MER-FND-001	Extension Lifecycle	Launch extension diagnostic workspace via Chromium toolbar action	Positive	Meridian unpacked extension (v0.1.0) is loaded into Chromium via Developer Mode. Background service worker (background.js) is registered and active.	Extension ID: chrome-extension://<id>/ Action: chrome.action.onClicked Target Page: index.html	1. Open Chromium browser with extension enabled. 2. Click the Meridian extension action button in browser toolbar. 3. Verify new browser tab creation. 4. Check URL and DOM structure of opened tab.	Browser instantiates and focuses a new tab navigated to chrome-extension://<id>/index.html. Page renders header "A local foundation for your research." and displays the "Embedding diagnostic" interface.	Tab successfully opened at chrome-extension://<id>/index.html; diagnostic interface and controls rendered immediately.	Pass	High	code/extension/src/background/index.ts#L2-L4
MER-FND-002	Diagnostic UI	Disable embedding trigger button for empty or whitespace-only textarea input	Negative	Meridian diagnostic tab is open. EmbeddingClient is initialized with status "Ready to load the packaged model."	Input A: "" (empty string) Input B: " " (whitespace characters only)	1. Open Meridian workspace in Chromium. 2. Clear textarea completely (Input A). 3. Inspect "Generate embedding" button disabled attribute. 4. Enter whitespace characters (Input B). 5. Inspect button disabled state and attempt mouse click.	"Generate embedding" button remains disabled (disabled={busy || !text.trim()}). Button is unclickable and no postMessage is dispatched to the background Web Worker. Status remains "Ready to load the packaged model."	Button remained strictly disabled; 0 worker messages dispatched; input rejected cleanly without UI flash.	Pass	Medium	code/extension/src/ui/main.tsx#L43-L44
MER-FND-003	Diagnostic UI	Enforce upper input boundary at MAX_INPUT_CHARACTERS (12,000 characters)	Boundary	Meridian diagnostic tab is open. MAX_INPUT_CHARACTERS contract constant is configured to 12,000.	Payload: String of exactly 12,000 valid characters generated by repeating "Meridian spatial knowledge engine. " followed by 50 additional overflow characters.	1. Navigate to diagnostic interface. 2. Attempt to paste 12,050 characters into the textarea. 3. Measure value length in DOM via textarea.value.length. 4. Click "Generate embedding". 5. Verify inference pipeline execution.	HTML textarea enforces maxLength={12000}, truncating input at exactly 12,000 characters. Inference executes successfully on the 12,000-character boundary payload without buffer overflow, returning a valid 384D vector.	Textarea truncated input at exactly 12,000 characters; inference executed to completion without browser memory warning.	Pass	High	code/extension/src/shared/contracts.ts#L4, code/extension/src/ui/main.tsx#L40
MER-FND-004	Embedding Engine	Generate normalized 384-dimensional dense semantic vector from valid text input	Positive	Meridian extension loaded in Chromium. Inference Web Worker active with Xenova/all-MiniLM-L6-v2 ONNX model.	Text: "A cat is sleeping on a warm windowsill." Character Count: 41 Backend: WASM CPU	1. Open Meridian diagnostic workspace (index.html). 2. Enter text into "Text to embed" textarea. 3. Click "Generate embedding" button. 4. Monitor UI lifecycle states: "Loading packaged model...", "Generating embedding...", and final output.	Inference succeeds in <500ms. Diagnostic UI displays dimensions=384, backend="wasm", elapsed time, and vector norm=1.000000. Vector contains exactly 384 finite float values with non-zero variance.	Vector generated with 384 finite values; vector norm = 1.000000; execution backend = wasm; latency = 360.2 ms (cold initialization).	Pass	High	code/extension/test-results/offline-verification.json
MER-FND-005	Embedding Engine	Verify warm-model inference performance acceleration on repeated calls	Positive	Model pipeline is already initialized and loaded in worker memory from prior execution.	Call 1 (Cold): "A cat is sleeping on a warm windowsill." Call 2 (Warm): "A kitten is resting beside the sunny window." Call 3 (Warm): "Database transactions provide atomicity and isolation."	1. Execute Call 1 (cold start) and record elapsed time. 2. Immediately enter Call 2 text and click "Generate embedding". 3. Record Call 2 elapsed time. 4. Immediately enter Call 3 text and record Call 3 elapsed time. 5. Compare execution latencies.	First call includes ONNX model and WASM initialization (~300–400ms). Subsequent calls reuse cached worker pipeline instance and execute in sub-15ms (<20ms SLA).	Call 1 latency = 360.2 ms; Call 2 latency = 7.7 ms; Call 3 latency = 7.6 ms (warm acceleration verified).	Pass	High	code/extension/test-results/offline-verification.json#L13-L17
MER-FND-006	Vector Space Topology	Verify semantic discrimination: related texts yield higher cosine similarity than unrelated texts	Positive	Meridian extension loaded in Chromium offline profile. Inference worker active.	Text A: "A cat is sleeping on a warm windowsill." Text B: "A kitten is resting beside the sunny window." Text C: "Database transactions provide atomicity and isolation."	1. Generate 384D embedding vector V_A for Text A. 2. Generate 384D embedding vector V_B for Text B. 3. Generate 384D embedding vector V_C for Text C. 4. Compute cosine similarity Cos(V_A, V_B) and Cos(V_A, V_C). 5. Compare separation delta: Cos(V_A, V_B) - Cos(V_A, V_C).	Conceptually related pair (cat/kitten) produces high positive cosine similarity (~0.589). Unrelated pair (cat/database) produces near-zero or negative similarity (~ -0.069). Separation delta exceeds 0.15.	Cos(V_A, V_B) = 0.588948; Cos(V_A, V_C) = -0.069150; separation delta = 0.658098 (> 0.15 threshold).	Pass	High	code/extension/tests/offline.spec.ts#L54-L58
MER-FND-007	Vector Space Topology	Verify mathematical identity boundary: identical inputs produce unit cosine similarity (1.000000)	Boundary	Embedding Web Worker is initialized and warm.	Prompt: "Meridian organizes research into a private, searchable knowledge map." (evaluated in two sequential runs).	1. Enter prompt and generate embedding vector V1. 2. Resubmit identical prompt without reloading and generate embedding vector V2. 3. Compute cosine similarity Cos(V1, V2) = (V1 . V2) / (||V1|| * ||V2||). 4. Check deviation from 1.0.	Cosine similarity between identical text embeddings evaluates to 1.000000 within strict numerical tolerance (+/- 0.00001). Vector generation is completely deterministic.	Cosine similarity = 1.000000; deterministic reproducibility confirmed across consecutive evaluations.	Pass	Medium	code/extension/tests/offline.spec.ts#L49-L52
MER-FND-008	Network Isolation	Guarantee zero external network requests during local embedding generation	Negative	Chromium launched in offline mode with blackhole proxy (127.0.0.1:9) and disabled DNS. Playwright network request listener attached.	Input: "Database transactions provide atomicity and isolation." Network Interceptor: All requests across context	1. Launch extension in isolated offline Chromium profile. 2. Set browser context to offline (context.setOffline(true)). 3. Attach request monitor collecting all dispatched URLs. 4. Trigger embedding generation in diagnostic UI. 5. Audit protocol and hostname of all intercepted URLs.	Zero external HTTP, HTTPS, or WebSocket requests observed. 100% of network traffic resolves exclusively to chrome-extension://<extensionId>/ origin. No data transmitted externally.	0 external requests observed; exactly 10 requests recorded, all resolving to local chrome-extension:// bundle.	Pass	High	code/extension/test-results/offline-verification.json#L18-L29
MER-FND-009	Network Isolation	Block external navigation probe in extension offline test environment	Negative	Chromium profile launched with proxy-server=http://127.0.0.1:9 and host-resolver-rules="MAP * ~NOTFOUND".	Target Probe URL: "https://example.com"	1. Launch offline test harness. 2. Create a new browser page within the persistent context. 3. Attempt page.goto("https://example.com", { timeout: 10000 }). 4. Catch and verify navigation exception.	Browser fails to establish external TCP connection; navigation request is blackholed/refused. Exception thrown confirming active network confinement.	Probe blocked; ERR_CONNECTION_REFUSED / NOTFOUND triggered; network isolation verified.	Pass	High	code/extension/tests/offline.spec.ts#L65-L70
MER-FND-010	Worker Security	Intercept and reject unauthorized external fetch calls from inside Web Worker	Negative	Inference Web Worker (embedding.worker.ts) is executing. Native fetch is wrapped with origin check.	Fetch attempt: fetch("https://huggingface.co/api/models") from inside worker context	1. Initialize Web Worker. 2. Attempt invocation of scope.fetch targeting an external domain. 3. Inspect returned Promise rejection reason.	Worker defense-in-depth wrapper rejects fetch with Error: "External request blocked: https://huggingface.co". Request is aborted prior to socket allocation.	Origin boundary check intercepted request; rejected with "External request blocked" error.	Pass	High	code/extension/src/workers/embedding.worker.ts#L13-L20
MER-FND-011	Asset Integrity	Detect tampered model binary via SHA-256 hash check and prevent corrupted execution	Negative	Model assets packaged in dist/models/. Exact SHA-256 hashes registered in assets.lock.json.	Corrupted binary: model_quantized.onnx modified with invalid byte payload.	1. Introduce 1-byte alteration into model_quantized.onnx. 2. Run asset integrity verification (npm run assets:prepare). 3. Launch extension and inspect diagnostic error state.	Verification detects SHA-256 hash mismatch against assets.lock.json. UI displays error alert: "Run npm run assets:prepare, rebuild, and reload the extension if assets are missing." Worker halts gracefully.	Asset checksum failure trapped; UI rendered role="alert" message; unhandled worker crash prevented.	Pass	High	code/extension/assets.lock.json, code/extension/src/ui/main.tsx#L47
MER-FND-012	Content Security Policy	Enforce strict MV3 CSP preventing inline script evaluation and unauthorized remote scripts	Negative	Extension manifest.json specifies content_security_policy.extension_pages.	CSP: "default-src 'self'; script-src 'self' 'wasm-unsafe-eval'; worker-src 'self'; connect-src 'self'; object-src 'none'; base-uri 'none'"	1. Load extension in Chromium. 2. Attempt to inject dynamic external script tag <script src="https://cdn.example.com/lib.js">. 3. Attempt to evaluate arbitrary string via eval(). 4. Audit browser console for security policy violations.	Chromium blocks script injection and eval() invocation with CSP Refusal. Only 'wasm-unsafe-eval' for local WebAssembly compilation and local packaged scripts are permitted.	CSP enforcement verified; external script and eval execution blocked by browser runtime.	Pass	High	code/extension/public/manifest.json#L9-L11
MER-FND-013	Worker Concurrency	Serialize rapid sequential inference requests via worker promise queue	Boundary	Meridian diagnostic interface is active with warm inference worker.	3 rapid consecutive postMessage events dispatched with unique request IDs (ID_1, ID_2, ID_3).	1. Dispatch Request 1 to Web Worker. 2. Without awaiting completion, immediately dispatch Request 2. 3. Immediately dispatch Request 3. 4. Observe execution order and response delivery.	Worker queue (queue = queue.then(() => embed(data))) serializes inference runs without concurrent session conflict or WASM memory corruption. All 3 promises resolve with their respective request IDs.	Queue serialized all 3 requests; outputs delivered to correct client map handlers without race conditions.	Pass	Medium	code/extension/src/workers/embedding.worker.ts#L71-L74
MER-FND-014	Worker Lifecycle	Terminate Web Worker cleanly and reject pending requests on client disposal	Positive	EmbeddingClient instance active with pending inference request in queue.	Pending request ID: crypto.randomUUID() Trigger: client.dispose()	1. Initiate embedding request. 2. Invoke client.dispose() while inference is pending (e.g. unmounting React root). 3. Check worker termination and promise status. 4. Attempt subsequent embed() call on disposed client.	Worker is immediately terminated (worker.terminate()). Pending request promise rejects with Error "Embedding client was closed.". Subsequent embed() calls immediately reject with "Embedding client is closed."	Worker killed; pending promises rejected cleanly; subsequent calls rejected without memory leak.	Pass	Medium	code/extension/src/workers/embedding-client.ts#L25-L42
MER-FND-015	Diagnostic UI	Display collapsible vector inspector with complete 384-element float array	Positive	Embedding generation completes successfully.	Output: EmbeddingResult with 384-element Float32Array.	1. Enter valid prose and click "Generate embedding". 2. Wait for completion and verify summary metrics (dimensions, backend, elapsed, norm). 3. Click <details> element titled "Inspect all vector values". 4. Inspect <pre data-testid="vector"> contents.	Details accordion expands to reveal full JSON-formatted array. Contains exactly 384 float numbers matching vector values. Vector norm equals hypot(...vector) to 6 decimal places.	Accordion revealed complete 384-element vector dump; data-testid="vector" parsed valid JSON array of length 384.	Pass	Low	code/extension/src/ui/main.tsx#L48-L58